Fakultas Informatika
School of Computing
Telkom University

Aplikasi Berbasis Platform

Layout (2)

Outline

- ListView
- GridView
- Stack
- Safe Area

ListView Class

- A scrollable list of widgets arranged linearly.
- ListView is the most commonly used scrolling widget. It displays its children one after another in the scroll direction. In the cross axis, the children are required to fill the ListView.
- If non-null, the itemExtent forces the children to have the given extent in the scroll direction.
- If non-null, the prototypeItem forces the children to have the same extent as the given widget in the scroll direction.

Four Options to Construct a ListView (1)

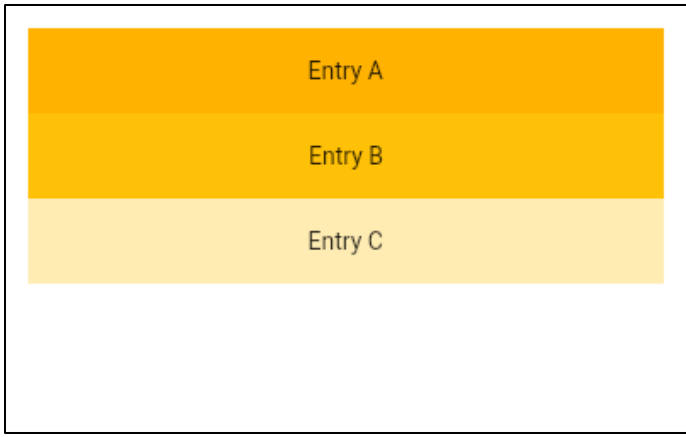
- The default constructor takes an explicit List<Widget> of children.
- The ListView.builder constructor takes an IndexedWidgetBuilder, which builds the children on demand.

Four Options to Construct a ListView (1)

- The ListView.separated constructor takes two IndexedWidgetBuilders: itemBuilder builds child items on demand, and separatorBuilder similarly builds separator children which appear in between the child items.
- The ListView.custom constructor takes a SliverChildDelegate, which provides the ability to customize additional aspects of the child model.

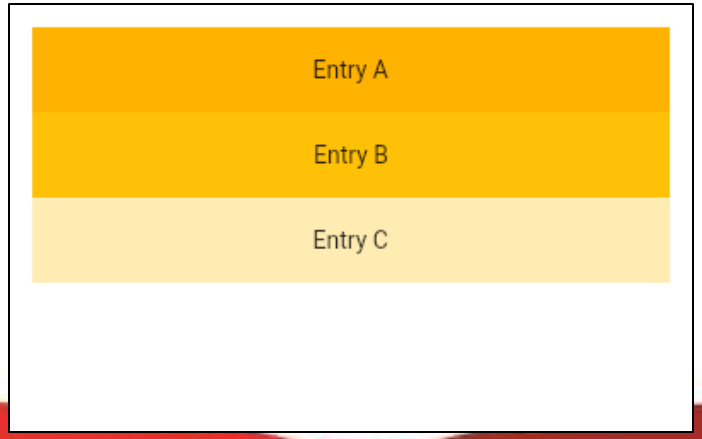
ListView Example (1)

```
ListView(
  padding: const EdgeInsets.all(8),
  children: <Widget>[
    Container(
      height: 50,
      color: Colors.amber[600],
      child: const Center(child: Text('Entry A')),
    ),
    Container(
      height: 50,
      color: Colors.amber[500],
      child: const Center(child: Text('Entry B')),
    ),
    Container(
      height: 50,
      color: Colors.amber[100],
      child: const Center(child: Text('Entry C')),
    ),
  ],
)
```



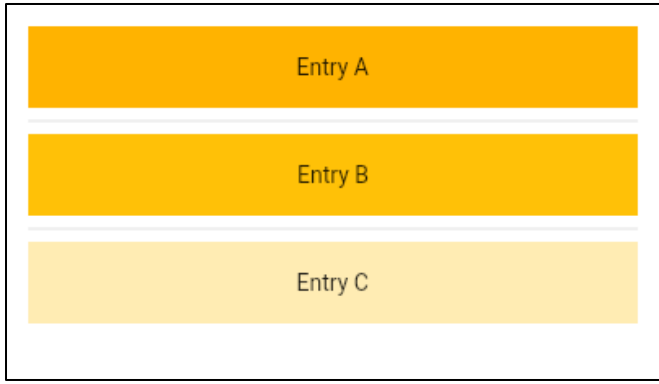
ListView Example (2)

```
final List<String> entries = <String>['A', 'B', 'C'];
final List<int> colorCodes = <int>[600, 500, 100];

ListView.builder(
  padding: const EdgeInsets.all(8),
  itemCount: entries.length,
  itemBuilder: (BuildContext context, int index) {
    return Container(
      height: 50,
      color: Colors.amber[colorCodes[index]],
      child: Center(child: Text('Entry ${entries[index]}')),
    );
  }
);
```



ListView Example (3)

```
final List<String> entries = <String>['A', 'B', 'C'];
final List<int> colorCodes = <int>[600, 500, 100];

ListView.separated(
  padding: const EdgeInsets.all(8),
  itemCount: entries.length,
  itemBuilder: (BuildContext context, int index) {
    return Container(
      height: 50,
      color: Colors.amber[colorCodes[index]],
      child: Center(child: Text('Entry ${entries[index]}')),
    );
  },
  separatorBuilder: (BuildContext context, int index) => const
Divider(),
);
```



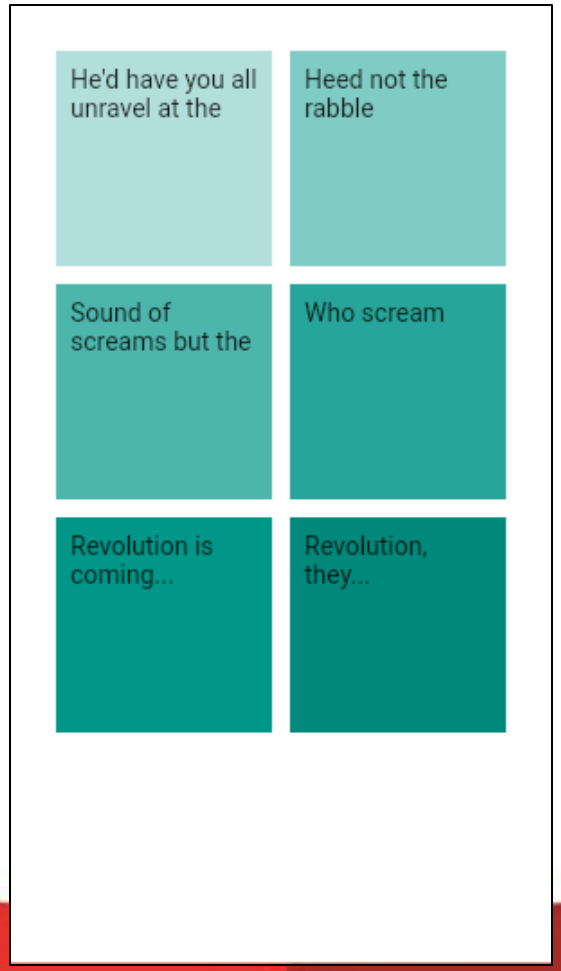
GridView Class

- The main axis direction of a grid is the direction in which it scrolls (the scrollDirection).
- The most commonly used grid layouts are GridView.count, which creates a layout with a fixed number of tiles in the cross axis, and GridView.extent, which creates a layout with tiles that have a maximum cross-axis extent.

GridView Example (1)

```
GridView.count(
  primary: false,
  padding: const EdgeInsets.all(20),
  crossAxisSpacing: 10,
  mainAxisSpacing: 10,
  crossAxisCount: 2,
  children: <Widget>[
    Container(
      padding: const EdgeInsets.all(8),
      child: const Text("He'd have you all unravel at the"),
      color: Colors.teal[100],
    ),
    Container(
      padding: const EdgeInsets.all(8),
      child: const Text('Heed not the rabble'),
      color: Colors.teal[200],
    ),
    …
  ],
)
```

GridView Output

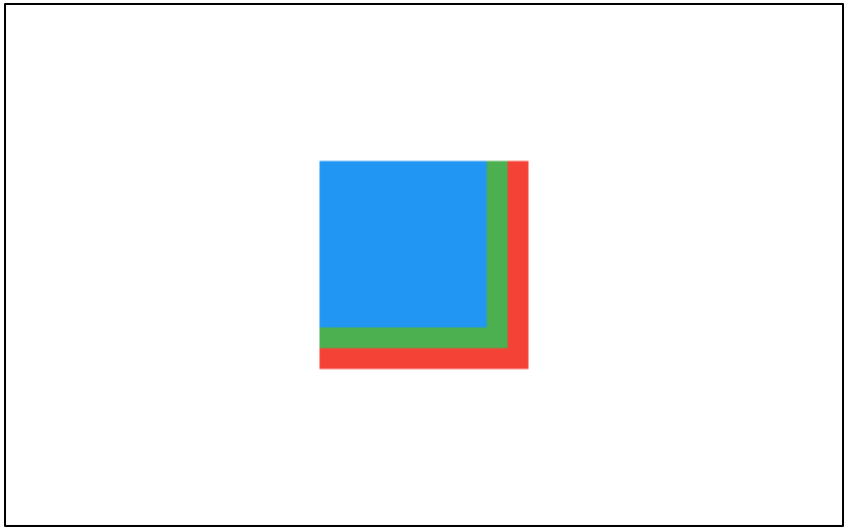


Stack

- A widget that positions its children relative to the edges of its box.
- This class is useful if you want to overlap several children in a simple way, for example having some text and an image, overlaid with a gradient and a button attached to the bottom.
- Each child of a Stack widget is either *positioned* or *non-positioned*. Positioned children are those wrapped in a Positioned widget that has at least one non-null property.

Stack Example (1)

```
Stack(
  children: <Widget>[
    Container(
      width: 100,
      height: 100,
      color: Colors.red,
    ),
    Container(
      width: 90,
      height: 90,
      color: Colors.green,
    ),
    Container(
      width: 80,
      height: 80,
      color: Colors.blue,
    ),
  ],
)
```

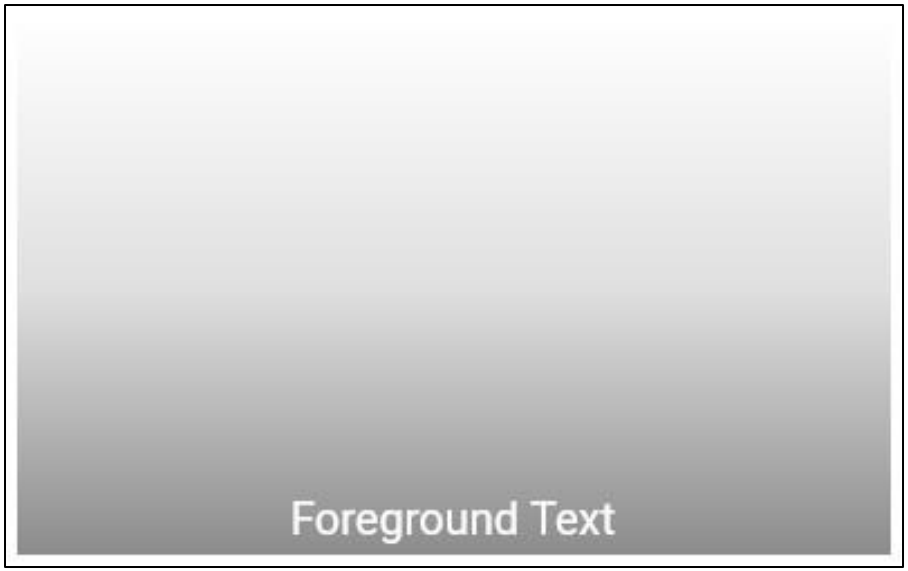


Stack Example (2)

```
SizedBox(
  width: 250,
  height: 250,
  child: Stack(
    children: <Widget>[
      Container(
        width: 250,
        height: 250,
        color: Colors.white,
      ),
      Container(
        padding: const EdgeInsets.all(5.0),
        alignment: Alignment.bottomCenter,
        decoration: BoxDecoration(
          gradient: LinearGradient(
            begin: Alignment.topCenter,
            end: Alignment.bottomCenter,
            colors: <Color>[
              ...
```

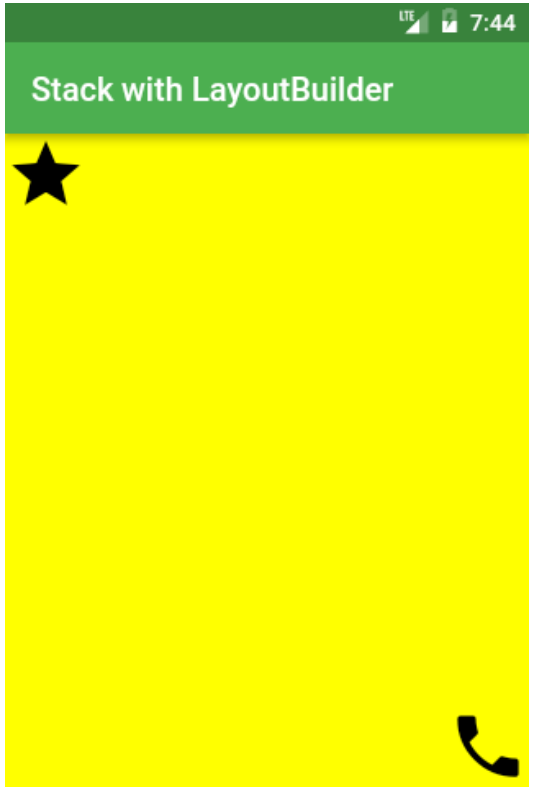
```
                    ...

    Colors.black.withAlpha(0),
              Colors.black12,
              Colors.black45
            ],
          ),
        ),
        child: const Text(
          'Foreground Text',
          style: TextStyle(color:
Colors.white, fontSize: 20.0),
        ),
      ),
    ],
  ),
)
```

Stack Example (2) Output



Stack Example (3)

```
LayoutBuilder(
  builder: (context, constraints) =>
    Stack(
      fit: StackFit.expand,
      children: <Widget>[
        Material(color: Colors.yellowAccent),
        Positioned(
          top: 0,
          child: Icon(Icons.star, size: iconSize),
        ),
        Positioned(
          top: constraints.maxHeight - iconSize,
          left: constraints.maxWidth - iconSize,
          child: Icon(Icons.call, size: iconSize),
        ),
      ],
    ),
),
```



Stack Example (4)

Safe Area

- *SafeArea* is an important and useful widget in Flutter which makes UI dynamic and adaptive to a wide variety of devices. While designing the layout of widgets, we consider different types of devices and their pre-occupied constraints of screen like status bar, notches, navigation bar, etc.

- In simple words, *SafeArea* is basically a padding widget, which adds any necessary padding to your app, based on the device it is running on. If your app's widgets are overlaying any of the system's features like notches, status bar, camera holes, or any other such features, then *SafeArea* would add padding around the app, as required

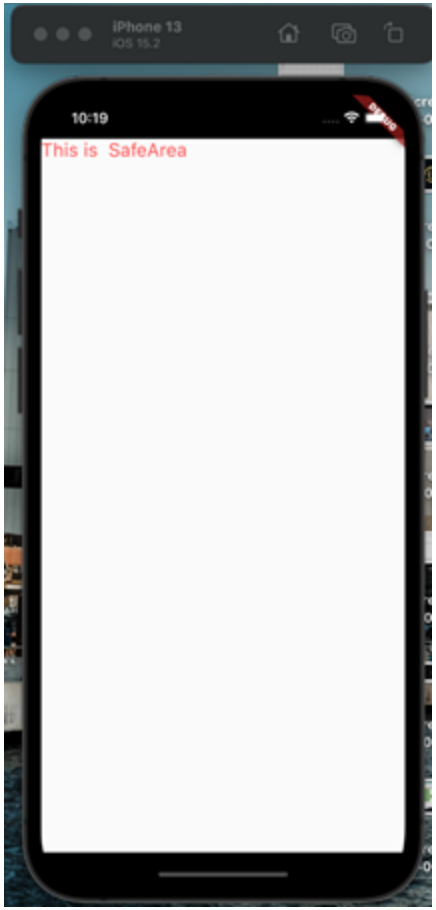
Safe Area(without)

This type takes a fixed list of children (pages) and makes them *scrollable*.

Code :

```
@override

  Widget build(BuildContext context) {

    return MaterialApp(

      home: Scaffold(

          body: Text(

            'This is Example without safe area',

            style: TextStyle(color: Colors.red, fontSize: 20),

          ),),);
```

Safe Area()



In the below code, we have used *SafeArea* and we can see that automatically padding is added around the text and is not covering the status bar.

Code :

```
@override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: SafeArea(
        top: true,
        child: Scaffold(
          body: Text(
            'This is SafeArea',
            style: TextStyle(color: Colors.redAccent, fontSize: 20),
          ),),),);
```

Any question?

References

- https://docs.flutter.dev/
- https://medium.com/flutter-community/flutter-layout-cheat-sheet-5363348d037e

Fakultas Informatika
School of Computing
Telkom University

THANK YOU